

UNITAT 5 · SETMANA 7

Canonades i composició de processos

📅 Setmana 7 · dates per confirmar </> Laboratori TD4 (resolt + repte IA) ~ 2 animacions

LABORATORI TD4 · RESULT

Canonades sense nom i composició de processos

Com construeix el shell `ls | grep | wc` : una canonada per cada `|`, un procés per ordre, `dup2` per endollar la sortida d'una a l'entrada de la següent i, sobretot, tancar a temps els extrems que no es fan servir.



Informe del laboratori

Entrega aquest laboratori amb un informe curt en LaTeX, amb la plantilla del curs: cada exercici explicat, el codi que importa i el que has executat al terminal. Pots fer servir IA generativa per escriure el LaTeX: declara-ho i verifica cada resultat. Plantilla, exemple i guia de Prism →

EXERCICIS

- | | | | |
|---|--|---|--------------------------------------|
| 1 | Una canonada entre pare i fill | 3 | <code>ps</code> en majúscules |
| 2 | Les regles de les canonades: capacitat, bloqueig i SIGPIPE | 4 | Un pipeline d'ordres fix |
| | | 5 | El pipeline des de la línia d'ordres |

TD4 · 1

● Resolt · provat

Una canonada entre pare i fill

Enunciat. Crea una canonada amb `pipe(2)` i duplica el procés. El pare escriu «Hello World» 10 vegades, avisa per la sortida d'error que se'n va i acaba. El fill llegeix com a màxim 500 bytes en una sola lectura, els imprimeix, ho avisa i acaba. Fes servir macros per a les constants.

Solució



C tds/td4/mypipe.c

```

/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a
 * child (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times
 * (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the
 * pipe capacity)
 * the parent blocks on write() once the pipe is full. When the child exits,
 * nobody can read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken
 * pipe", status 141
 * in the shell). That only works if the parent CLOSED its own read end: with
 * -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a
 * potential reader,
 * the kernel never raises SIGPIPE and the parent blocks forever.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE  "Hello World\n"
#define READ_MAX 500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end,
    fds[1] write end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                             /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
    }
}

```

```

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
    fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
}

/* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");           /* THE line that
exercise 2 is about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n",
count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave

```

`fds[0]` és l'extrem de lectura i `fds[1]` el d'escriptura (com 0 = entrada i 1 = sortida). Després de `fork` tots dos processos tenen els dos extrems: cadascun tanca el que no fa servir.

Les regles de les canonades: capacitat, bloqueig i SIGPIPE

Enunciat. Passa com a argument quantes vegades escriu el pare: 50, 100, 500, 1000, 5000, 10000... Si el programa sembla penjat, comprova l'estat dels processos, dibuixa les taules de descriptors i corregeix-lo. Amb el programa corregit, quin és el seu estat de sortida?

Solució

Sortida real

```
$ python3 -c 'import os, fcntl; r, w = os.pipe(); print(fcntl.fcntl(w,
fcntl.F_GETPIPE_SZ), "bytes")'
65536 bytes
```

Sortida real

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $?" (124 =
killed by timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Sortida real

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"
I am the child, I read 500 bytes and I leave
exit status of mypipe: 141
```

Nou: la cursa, mesurada. L'enunciat original proposava trobar «a partir de quants missatges» mor el pare. En una màquina multinucli no hi ha cap llindar fix: el fill llegeix 500 bytes i se'n va mentre el pare encara escriu, i qui guanya depèn del planificador. 30 execucions per mida:

Sortida real

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses;
done
  50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 100 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 200 messages: 28 x status 0      2 x status 141 (SIGPIPE)
 300 messages: 19 x status 0     11 x status 141 (SIGPIPE)
 400 messages:  5 x status 0     25 x status 141 (SIGPIPE)
 500 messages:  5 x status 0     25 x status 141 (SIGPIPE)
1000 messages:  0 x status 0     30 x status 141 (SIGPIPE)
```

Mira-ho animat → A5.2

1. La canonada té una capacitat limitada: **65.536 bytes** per defecte a Linux. $10.000 \times 12 \text{ bytes} = 120.000 \text{ bytes}$ no hi caben: quan s'omple, `write` *bloqueja* el pare fins que algú llegeix.
2. El fill llegeix 500 bytes i acaba. Si el pare **no ha tancat** el seu propi extrem de lectura (`mypipe_bug`), el nucli veu que encara hi ha un lector —el mateix pare— i el deixa bloquejat per sempre: `timeout` el mata al cap de 3 s (estat 124).
3. Amb l'extrem tancat, quan el fill se'n va no queden lectors: el `write` següent rep el senyal **SIGPIPE**, l'acció per defecte del qual és acabar el procés. El shell mostra «Broken pipe» i l'estat $141 = 128 + 13$.

Animació A5.2 · Capacitat de la canonada →

TD4 · 3

● Result · provat

ps en majúscules

Enunciat. El fill llegeix contínuament de la canonada i imprimeix en majúscules el que llegeix. El pare redirigeix la seva sortida estàndard a la canonada amb `dup2` i executa `ps` amb `exec lp`.

Solució



C tds/td4/upper.c

```

/*
 * TD4 - exercise 3: ps | uppercase, built by hand.
 * The child reads the pipe until end of file and prints it in uppercase.
 * The parent redirects its stdout to the pipe (dup2) and becomes `ps`
 (execlp).
 * The child sees end of file when `ps` exits, because it was the last
 writer.
 */
#include <ctype.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define BUFFER_SIZE 512

static void to_upper(int fd_in, int fd_out)
{
    char buffer[BUFFER_SIZE];
    ssize_t n;

    while ((n = read(fd_in, buffer, sizeof buffer)) > 0) {    /* until end
of file */
        for (ssize_t i = 0; i < n; i++)
            buffer[i] = (char)toupper((unsigned char)buffer[i]);
        exit_if(write_all(fd_out, buffer, (size_t)n) == -1, "write");
    }
    exit_if(n == -1, "read");
}

int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                                           /* child */
        exit_if(close(fds[1]) == -1, "close");               /* otherwise it never sees
end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }
}

```

```

}

/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");      /* 1 already refers to the
pipe */
execlp("ps", "ps", (char *)NULL);          /* "ps" twice: file to run,
then argv[0] */
exit_if(1, "execlp ps");
return EXIT_FAILURE;
}

```

Sortida real

```

$ ./upper | head -4
  PID TTY          TIME CMD
  6706 ?           00:00:25 UVICORN
  9771 ?           00:00:03 SYSTEMD
  9801 ?           00:00:00 (SD-PAM)

```

El fill acaba quan `read` retorna 0, i això només passa quan tots els extrems d'escriptura són tancats: el de `ps` (quan acaba) i el seu, i per això el tanca al principi. `ps` no sap que escriu en una canonada: el seu descriptor 1 simplement hi apunta.

TD4 · 4

● Result · provat

Un pipeline d'ordres fix

Enunciat. Executa el pipeline definit en una variable `commands` (per exemple `ls -l | tr -d '[:blank:]'`), amb qualsevol nombre d'ordres. El programa acaba quan acaba l'última i en retorna l'estat; si ha mort per un senyal, ho informa i retorna `EXIT_FAILURE`.

Solució



C tds/td4/pipeline.c

```

/*
 * TD4 - exercise 4: run a fixed pipeline, like the shell does for ls -l |
 * tr -d '[:blank:]'
 * Works for any number of commands. Returns the exit status of the LAST
 * command, or
 * EXIT_FAILURE (with a message) if it was killed by a signal.
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

static char *ls[] = { "ls", "-l", NULL };
static char *tr[] = { "tr", "-d", "[:blank:]", NULL };
static char **commands[] = { ls, tr };
static const int ncommands = sizeof commands / sizeof commands[0];

int main(void)
{
    int in = STDIN_FILENO;      /* where the next command reads from */
    pid_t last = -1;

    for (int i = 0; i < ncommands; i++) {
        int fds[2] = { -1, -1 };
        if (i < ncommands - 1) /* every command but
the last writes to a pipe */
            exit_if(pipe(fds) == -1, "pipe");

        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            if (in != STDIN_FILENO) { /* not the first: read
from previous pipe */
                exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
                exit_if(close(in) == -1, "close");
            }
            if (fds[1] != -1) { /* not the last: write
to the new pipe */
                exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
                exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
            }
        }
    }
}

```

```

        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }

    /* parent: close what the child inherited, keep only the read end for
    the next one */
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}

/* Wait for every child (no zombies), remember the status of the last
one. */
int status, last_status = 0;
pid_t pid;
while ((pid = wait(&status)) > 0)
    if (pid == last)
        last_status = status;

if (WIFEXITED(last_status))
    return WEXITSTATUS(last_status);
if (WIFSIGNALED(last_status))
    fprintf(stderr, "pipeline: last command killed by %s\n",
    signame(WTERMSIG(last_status)));
    return EXIT_FAILURE;
}

```

Sortida real

```

$ cd demo && ../pipeline
total 8
-rw-rw-r--1 student student 4 Sep 30 13:13 one.txt
-rw-rw-r--1 student student 2 Sep 30 13:13 two.txt

```

El patró: cada volta crea la canonada *següent* (excepte per a l'última ordre); el fill pren la canonada anterior (**in**) com a entrada i la nova com a sortida; el pare tanca el que el fill ja ha heretat i guarda l'extrem de lectura per a la volta següent. Al final el pare espera **tots** els fills (si no, quedarien zombis) i guarda l'estat de l'últim.

El pipeline des de la línia d'ordres

Enunciat. Com l'anterior, però les ordres arriben per la línia d'ordres separades per `--`:
`pipeline ls -l -- grep a -- wc -l` equival a `ls -l | grep a | wc -l`.

Solució

C tds/td4/pipeline_args.c

```
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s cmd [args] [-- cmd [args]]...\n",
argv[0]);
        return EXIT_FAILURE;
    }

    /* Split argv in place. commands[k] points to the first word of command
k. */
    char **commands[argc];
    int ncommands = 0;
    commands[ncommands++] = &argv[1];
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            argv[i] = NULL; /* ends the previous
command */
            if (i + 1 ≥ argc || strcmp(argv[i + 1], "--") == 0 ||
commands[ncommands - 1][0] == NULL) {
                fprintf(stderr, "%s: empty command\n", argv[0]);
                return EXIT_FAILURE;
            }
            commands[ncommands++] = &argv[i + 1];
        }
    }
}
```

Sortida real

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Sortida real

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Sortida real

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Truc: `argv` ja és un vector de cadenes acabat en `NULL`. Substituint cada `--` per `NULL`, cada ordre esdevé un tros d'`argv` a punt per a `execvp`, sense copiar res. La resta és idèntica a l'exercici 4.

Llista de comprovació

Repte amb IA generativa · Canonades al teu mini shell



Objectiu. Afegeix l'operador `|` al mini shell reutilitzant `pipeline_args.c`, i diagnostica amb la IA un bloqueig provocat a propòsit.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Tinc un mini shell i un programa que executa pipelines de N ordres [enganxa'ls tots dos]. (1) Ajuda'm a integrar l'operador `|` al shell. (2) Després trauré a propòsit un `close()` del pare: explica, dibuixant les taules de descriptors de cada procés, per què `ls | wc -l` ja no acaba, i com ho confirmaries amb `ls -l /proc/<pid>/fd`. No corregeixis tu el codi: guia'm amb preguntes.

Com comprovar el resultat

- `ls -l | grep c | wc -l` dona el mateix resultat que `bash`.
- Sense el `close()`, l'ordre es queda penjada i `/proc/<pid>/fd` mostra l'extrem d'escriptura encara obert.
- Saps dir quin procés tenia el descriptor que impedia el final de fitxer.

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.